ПМ3 Разработка модулей ПО.

РО 3.1 Понимать и применять принципы объектно-ориентированного и асинхронного программирования.

Тема 2. Асинхронно программирование.

Лекция 16. fetch с async/await и обработка ошибок.

Цель занятия:

Показать, как работают асинхронные HTTP-запросы в JavaScript, разобрать инструменты **fetch** и **XMLHttpRequest**, их связь с промисами и использование в реальных приложениях.

Учебные вопросы:

1. Использование async/await для HTTP-запросов
2. Сетевые ошибки и статус ответа
3. Практические примеры: работа с публичными API
4. Типичные ошибки при работе с fetch

1. Использование async/await для HTTP-запросов

Зачем использовать async/await?

- Работа с fetch через .then() и .catch() может делать код вложенным и менее читаемым.
- async/await позволяет писать асинхронный код в пошаговом стиле, похожем на обычный синхронный.
- Это упрощает логику и особенно полезно, когда нужно сделать несколько запросов подряд.

Пример:

```
async function getPost() {
  try {
    let response = await fetch("https://jsonplaceholder.typicode.com/posts/1");
    let data = await response.json(); // ждём, пока ответ превратится в JSON
    console.log("Пост:", data);
  } catch (error) {
    console.error("Ошибка сети:", error);
  }
}

getPost();
```

📌 Здесь:

- await fetch(...) ждёт завершения запроса.
- await response.json() ждёт преобразования ответа в JSON.
- Ошибки (например, если нет интернета) ловятся в try/catch.

Результат:

```
[Running] node "c:\Users\user\Documents\JScode\test1\test.js"
Пост: {
  userId: 1,
  id: 1,
  title: 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit',
  body: 'quia et suscipit\n' +
    'suscipit recusandae consequuntur expedita et cum\n' +
    'reprehenderit molestiae ut ut quas totam\n' +
    'nostrum rerum est autem sunt rem eveniet architecto'
}
```

Проверка успешности ответа.

Важно помнить: fetch не выбрасывает ошибку при кодах 404 или 500.

Поэтому нужно проверять response.ok.

📌 Здесь при 404 запрос дойдёт до throw, и управление перейдёт в catch.

```
async function getData() {
  try {
    let response = await fetch("https://jsonplaceholder.typicode.com/unknown");
    if (!response.ok) {
      throw new Error("Ошибка HTTP: " + response.status);
    }
    let data = await response.json();
    console.log("Данные:", data);
  } catch (error) {
    console.error("Запрос не удался:", error.message);
  }
}
getData();
```

Последовательные запросы.

Если нужно выполнить несколько запросов один за другим:

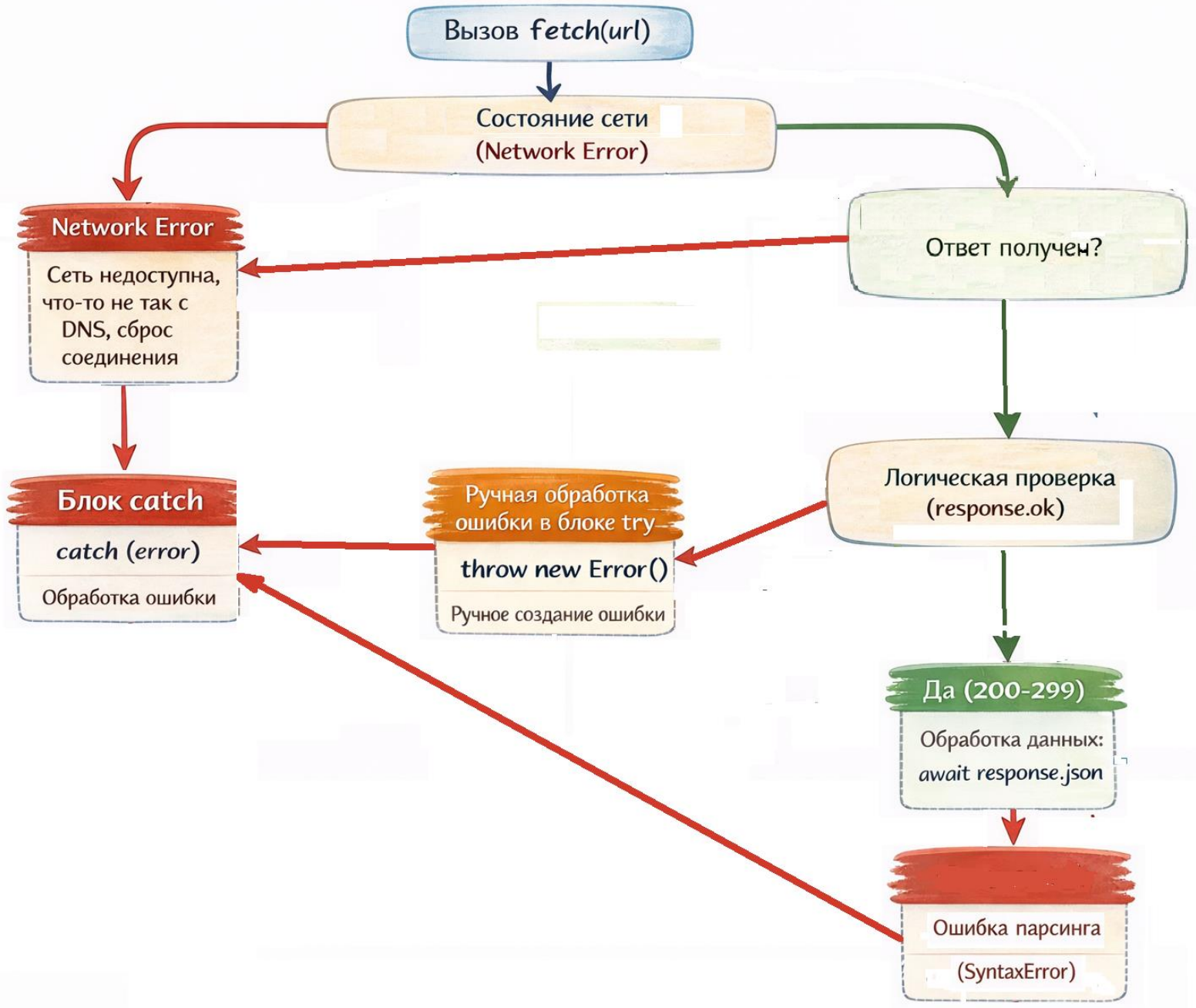
```
async function getPostAndUser() {
  try {
    let postRes = await fetch("https://jsonplaceholder.typicode.com/posts/1");
    let post = await postRes.json();
    let userRes = await fetch("https://jsonplaceholder.typicode.com/users/"
        + post.userId);
    let user = await userRes.json();
    console.log("Пост:", post.title);
    console.log("Автор:", user.name);
  } catch (e) {
    console.error("Ошибка:", e);
  }
}
getPostAndUser();
```

Параллельные запросы.

Если запросы независимы, их лучше выполнять параллельно через Promise.all. Оба запроса выполняются одновременно, что экономит время.

```
async function getParallel() {
  try {
    let [postsRes, usersRes] = await Promise.all([
      fetch("https://jsonplaceholder.typicode.com/posts"),
      fetch("https://jsonplaceholder.typicode.com/users")
    ]);
    let posts = await postsRes.json();
    let users = await usersRes.json();
    console.log("Всего постов:", posts.length);
    console.log("Всего пользователей:", users.length);
  } catch (e) {
    console.error("Ошибка:", e);
  }
}
getParallel();
```

⚡ Жизненный цикл **fetch**-запроса



Вывод:

- async/await делает работу с fetch более простой и наглядной.
- Можно использовать try/catch для перехвата ошибок.
- Для последовательных операций код выглядит линейным.
- Для параллельных запросов удобно сочетать await с Promise.all.

2. Сетевые ошибки и статус ответа.

При работе с fetch и HTTP-запросами можно столкнуться с двумя основными типами ошибок:

- Сетевые ошибки — когда запрос вообще не дошёл до сервера (например, нет интернета, неверный домен).
- Ошибки статуса ответа — сервер ответил, но с кодом, который указывает на проблему (404 — «не найдено», 500 — «внутренняя ошибка»).

Сетевые ошибки.

Если сеть недоступна или сервер «не отвечает», промис fetch переходит в reject, и ошибка ловится в .catch() или try/catch.

👉 Пример:

```
async function networkErrorDemo() {
  try {
    let response = await fetch("https://wrong-domain.example.com/data");
    let data = await response.json();
    console.log(data);
  } catch (err) {
    console.error("Сетевая ошибка:", err.message);
  }
}
networkErrorDemo();
```

```
[Running] node "c:\Users\user\Documents\JScode\test1\test.js"
Сетевая ошибка: fetch failed
```

📌 Здесь catch перехватит ошибку (например, «Failed to fetch»).

Ошибки статуса ответа.

Важно: fetch не выбрасывает ошибку, если сервер вернул 404 или 500.

Вместо этого нужно вручную проверить свойства ответа:

- response.ok → true, если статус в диапазоне 200–299.
- response.status → сам код ответа.

Пример:

```
async function statusErrorDemo() {
  let response = await fetch("https://jsonplaceholder.typicode.com/unknown");
  if (!response.ok) {
    console.error("Ошибка HTTP:", response.status); // 404
    return;
  }
  let data = await response.json();
  console.log(data);
}
statusErrorDemo();
```

📌 В этом примере сервер вернёт 404, поэтому данные не будут обработаны.

Распространённые коды ответов:

- 200 — OK (успех, данные получены).
- 201 — Created (новый ресурс создан, например при POST-запросе).
- 400 — Bad Request (ошибка клиента, неправильный запрос).
- 401 — Unauthorized (нужна авторизация).
- 403 — Forbidden (нет прав доступа).
- 404 — Not Found (ресурс не найден).
- 500 — Internal Server Error (ошибка на стороне сервера).

Все коды: https://http.cat/

Комбинированный подход.

Можно объединить проверку сети и статуса в одном блоке try/catch:

```
async function safeFetch() {
  try {
    let response = await fetch("https://jsonplaceholder.typicode.com/unknown");
    if (!response.ok) {
      throw new Error("Ошибка HTTP: " + response.status);
    }
    let data = await response.json();
    console.log("Данные:", data);
  } catch (error) {
    console.error("Запрос не удался:", error.message);
  }
}
safeFetch();
```

📌 Здесь:

- Если нет интернета → попадём в catch.
- Если сервер вернул 404 → сработает throw, и тоже попадём в catch.

✅ **Вывод:**

- fetch может завершиться ошибкой из-за проблем с сетью.
- Ошибки HTTP (404, 500) не считаются сетевыми — их нужно проверять через response.ok.
- Хороший стиль: всегда оборачивать запросы в try/catch и проверять response.status.

3. Практические примеры: работа с публичными API.

При работе с fetch и HTTP-запросами можно столкнуться с двумя основными типами ошибок:

- Сетевые ошибки — когда запрос вообще не дошёл до сервера (например, нет интернета, неверный домен).
- Ошибки статуса ответа — сервер ответил, но с кодом, который указывает на проблему (404 — «не найдено», 500 — «внутренняя ошибка»).

Например:

1. Курсы валют. ExchangeRate API: exchangerate-api.com
2. Криптовалюты. CoinGecko API: coingecko.com/en/api
3. Погода. OpenWeatherMap: openweathermap.org/api
4. Случайные факты. Random Facts API: uselessfacts.jsph.pl
5. Цитаты. Quotable API: quotable.io
6. Генерация случайных данных. randomuser.me
7. Данные о животных.

Dog API: dog.ceo/dog-api

Cat API: thecatapi.com.

8. Рецепты. Recipe API: spoonacular.com/food-api
9. Факты о стране. REST Countries API: restcountries.com

Курсы валют — ExchangeRate API

```
async function getRates() {
  let res = await fetch("https://open.er-api.com/v6/latest/USD");
  let data = await res.json();
  console.log("Курс USD к EUR:", data.rates.EUR);
  console.log("Курс USD к JPY:", data.rates.JPY);
}

getRates();
```

```
[Running] node "c:\Users\user\Documents\JScode\test1\test.js"
Курс USD к EUR: 0.853717
Курс USD к JPY: 147.548586
```

Данные о стране — REST Countries API

```javascript
async function getCountry() {
  let res = await fetch("https://restcountries.com/v3.1/name/qazaqstan");
  let data = await res.json();
  console.log("Страна:", data[0].name.common);
  console.log("Столица:", data[0].capital[0]);
  console.log("Регион:", data[0].region);
}

getCountry();
```

```
Страна: Kazakhstan
Столица: Astana
Регион: Asia
```

4. Типичные ошибки при работе с fetch».

1. Ожидание «автоматической» ошибки при 404/500

Самая частая ошибка новичков. Они думают, что если сервер ответил «404 Not Found», то код сразу перейдет в блок catch.

Реальность: fetch выбрасывает исключение (переходит в catch) только если запрос не удалось выполнить физически (отключен интернет, неверный домен, блокировка CORS).

Решение: Всегда проверять свойство response.ok (коды 200–299) или response.status.

2. Забытый `await` при вызове `.json()`

Студенты часто пишут `const data = response.json();` и удивляются, почему в data лежит объект Promise { <pending> }, а не данные.

Реальность: Чтение тела ответа из потока и превращение его в JSON — это процесс, требующий времени.

Решение: Использовать `const data = await response.json();`

3. Попытка прочитать тело ответа дважды

Поток данных в fetch можно прочитать только один раз.

Пример ошибки:

```
console.log(await response.text()); // Прочитали как текст
const data = await response.json(); // Ошибка! Тело ответа уже пустое.
```

Решение: Если данные нужны в разном виде, сначала сохраните результат парсинга в переменную или используйте метод response.clone().

4. Проблемы с CORS (Cross-Origin Resource Sharing)

Часто при работе с публичными API студенты видят ошибку в консоли: «Access to fetch at... has been blocked by CORS policy».

Суть: Это не ошибка кода, а механизм безопасности браузера. Сервер должен разрешить вашему домену запрашивать данные.

Решение: это решается настройкой заголовков на сервере или использованием прокси-серверов для разработки.

5. Отсутствие обработки ошибок сети

Если просто написать fetch, но не обернуть его в try/catch, то при внезапном обрыве связи приложение «упадет».

Решение: Любой сетевой запрос должен быть обернут в блок try...catch.

Резюме (Short-list):

- Ошибка: Забыл try/catch — приложение падает при отсутствии сети.
- Ошибка: Забыл if (!response.ok) — код думает, что всё хорошо, хотя пришла 404 ошибка.
- Ошибка: Забыл await перед .json() — получил «промис» вместо данных.
- Ошибка: Дважды вызвал .json() — получил ошибку "body used".

Итоги лекции:

- fetch() всегда возвращает Promise. Это современный стандарт, который пришел на смену перегруженному XMLHttpRequest.
- Использование async/await не меняет логику работы fetch, но делает код «плоским» и читаемым, избавляя от вложенных .then().
- Нужно помнить, что await требуется дважды: первый раз для получения самого ответа (Response), второй — для чтения и парсинга его тела (например, через .json()).
- fetch перейдет в блок catch только при сетевом сбое (нет интернета, ошибка DNS). Ошибки сервера (404, 500) считаются «успешно доставленным ответом».
- Свойство response.ok: Это главный индикатор успеха. Оно равно true только в диапазоне HTTP-статусов 200–299.
- Потоки данных: Тело ответа (body) является потоком, поэтому его можно прочитать только один раз.

Контрольные вопросы:

- Почему при использовании fetch с async/await нам всё равно нужен блок try...catch?
- Что именно возвращает вызов fetch('https://api.example.com')?
- Зачем нужно проверять response.ok, если запрос уже выполнился без ошибок в блоке try?
- В какой блок (try или catch) попадет выполнение, если сервер вернул ошибку 500 Internal Server Error? Почему?
- Нужно ли ставить await перед response.json()? Что произойдет, если этого не сделать?
- Как ведут себя элементы интерфейса (кнопки, анимации) в момент, когда fetch ожидает ответа от сервера? (Связь с Event Loop).
- Что произойдет, если вызвать await response.json() дважды для одного и того же объекта ответа?
- Представьте, что API вернуло статус 404. Как принудительно отправить выполнение кода в блок catch?
- В чем разница между обработкой ошибки сети и обработкой ошибки бизнес-логики (например, «Пользователь не найден»)?

Домашнее задание:

https://ru.hexlet.io/programs/js-async

Материалы лекций:

https://github.com/ShViktor72/Education2025

Обратная связь:

colledge20education23@gmail.com